

C语言指针与链表专项训练 - Day 5（薄弱点强化版）

参考：谭浩强《C程序设计》

- 题型：单项选择题，共50题，每题只有一个正确答案
- 重点：指针与链表，并综合函数、循环、输入输出等内容
- 形式：以完整或较完整的标准C程序阅读、运行结果分析和错误定位为主
- 约定：除题目特别说明外，动态内存申请均成功
- 本卷针对Day4错题和疑问题设计全新变式，不原样重复Day1—Day4题目
- 本卷不涉及时间复杂度，试卷不附答案

答题说明

建议不运行程序独立完成。若某题不确定，请在答案后加 `?`，例如 `8.c?`；即使选择正确，批改时也会重点讲解。

输出选项忽略行末多余空格。

一、动态内存与指针别名

1. 下列程序的输出是：

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Item {
    int count;
    char tag;
} Item;

int main(void)
{
    Item *p = calloc(3, sizeof *p);

    p[1].count = 4;
    p[2].tag = 'A';
    printf("%d %d %d",
           p[0].count, p[1].count, p[2].tag == 'A');

    free(p);
    return 0;
}
```

- A. 不确定 4 1
- B. 0 4 1
- C. 0 0 1
- D. 0 4 65

2. 下列程序存在的主要问题是：

```
#include <stdlib.h>

int main(void)
{
    int *base = malloc(4 * sizeof *base);
    int *middle = base + 1;
    int i;

    for (i = 0; i < 4; i++)
        base[i] = i + 1;

    free(middle);
    return 0;
}
```

- A. `base + 1` 不能赋给指针
- B. 循环访问了数组范围之外的元素
- C. `free` 的参数不是内存分配函数返回的起始地址
- D. 动态数组不能保存整数

3. 下列程序的输出是：

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *p = malloc(2 * sizeof *p);
    int *q;
    int i, sum = 0;

    p[0] = 4;
    p[1] = 6;
    q = realloc(p, 4 * sizeof *q);
    q[2] = 8;
    q[3] = 10;

    for (i = 0; i < 4; i++)
        sum += q[i];
    printf("%d %d", sum, q[0] + q[1]);

    free(q);
    return 0;
}
```

- A. 28 10
- B. 18 10
- C. 28 14
- D. 原来的两个元素一定丢失，输出不确定

4. 假设下列程序中的 `realloc` 成功，正确的说法是：

```
#include <stdlib.h>

int main(void)
{
    int *p = malloc(6 * sizeof *p);
    int *inside;
    int *r;
    int i;

    for (i = 0; i < 6; i++)
        p[i] = i + 10;

    inside = p + 2;
    r = realloc(p, 3 * sizeof *r);

    /* 后续操作 */
    free(r);
    return 0;
}
```

- A. `inside` 仍可安全解引用，因为它原来指向前三个元素之一
- B. 程序结束前还应分别执行 `free(p)` 和 `free(inside)`
- C. `r[0]` 至 `r[2]` 保留原值，但后续不能再通过 `p` 或 `inside` 访问原对象
- D. `realloc` 只能改变空间大小，返回地址一定等于 `p`

5. 下列程序的输出是：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void)
{
    const char *left = "pointer";
    const char *right = "list";
    char *s = malloc(strlen(left) + strlen(right) + 1);
    char *p;

    strcpy(s, left);
    strcat(s, right);
    p = strchr(s, 'l');

    printf("%s %td", s, p - s);
    free(s);
    return 0;
}
```

- A. pointer 7
- B. pointerlist 6
- C. pointerlist 8
- D. pointerlist 7

二、字符串内部指针与重叠内存移动

6. 下列程序的输出是：

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[] = "abracadabra";
    char *first = strchr(s, 'a');
    char *second = strchr(first + 1, 'a');

    *second = 'A';
    printf("%s %td", s, second - s);
    return 0;
}
```

- A. Abracadabra 0
- B. abrAcadabra 3
- C. abracAdabra 5
- D. abracadabrA 10

7. 下列程序的输出是：

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[] = "level-headed";
    char *p = strrchr(s, 'e');

    printf("%td %c %c", p - s, p[-1], p[1]);
    return 0;
}
```

- A. 10 d d
- B. 7 h a
- C. 3 v l

- D. 1 1 v

8. 下列程序的输出是：

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[12] = "ABCDE";
    char *p = strchr(s, 'C');

    memmove(p, p + 1, strlen(p + 1) + 1);
    printf("%s %c", s, *p);
    return 0;
}
```

- A. ABCDE C
- B. ABCD D
- C. ABDE D
- D. 由于源区域和目标区域重叠，行为未定义

9. 下列程序的输出是：

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[12] = "code";
    char *p = s + 1;

    memmove(p + 2, p, strlen(p) + 1);
    p[0] = 'X';
    p[1] = 'Y';

    printf("%s", s);
    return 0;
}
```

- A. cXYde

- B. XYcode
- C. coXYde
- D. cXYode

10. 下列程序的输出是：

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[10] = "12345";
    char *p = s + 2;

    memmove(s + 1, s, strlen(s) + 1);
    printf("%s %c %td", s, *p, p - s);
    return 0;
}
```

- A. 112345 3 2
- B. 112345 2 2
- C. 12345 3 2
- D. 112345 2 3

三、函数指针与回调函数

11. 下列程序的输出是：

```
#include <stdio.h>

int plus2(int x)
{
    return x + 2;
}

int triple(int x)
{
    return x * 3;
}

int half(int x)
{
    return x / 2;
}

int main(void)
{
    int (*ops[3])(int) = {triple, plus2, half};
    int order[] = {1, 0, 2};
    int x = 4;
    int i;

    for (i = 0; i < 3; i++)
        x = ops[order[i]](x);

    printf("%d", x);
    return 0;
}
```

- A. 9
- B. 7
- C. 10
- D. 18

12. 下列程序的输出是：

```
#include <stdio.h>

int add_key(int x, int key)
{
    return x + key;
}

int multiply_key(int x, int key)
{
    return x * key;
}

void transform(int a[], int n, int key,
               int (*operation)(int, int))
{
    int i;

    for (i = 0; i < n; i++)
        a[i] = operation(a[i], key);
}

int main(void)
{
    int a[] = {1, 2, 3};

    transform(a, 3, 2, add_key);
    transform(a, 3, 3, multiply_key);
    printf("%d %d %d", a[0], a[1], a[2]);
    return 0;
}
```

- A. 5 7 9
- B. 3 4 5
- C. 9 12 15
- D. 6 12 18

13. 下列程序的输出是：

```
#include <stdio.h>

typedef int (*Operation)(int, int);

int add(int x, int y)
{
    return x + y;
}

int subtract(int x, int y)
{
    return x - y;
}

Operation choose(int mode)
{
    if (mode > 0)
        return add;
    return subtract;
}

int main(void)
{
    Operation f = choose(0);
    Operation g = choose(1);

    printf("%d %d", f(9, 4), g(9, 4));
    return 0;
}
```

- A. 13 5
- B. 5 13
- C. 5 5
- D. 13 13

四、逗号运算符、指针自增与数组形参

14. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[] = {2, 4, 6};
    int *p = a;
    int x = (p++, *p);
    int y = ((*p)++, *p);

    printf("%d %d %d %td", x, y, a[1], p - a);
    return 0;
}
```

- A. 2 4 5 0
- B. 4 4 5 1
- C. 4 5 4 1
- D. 4 5 5 1

15. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[] = {2, 4, 6};
    int *refs[] = {a + 2, a, a + 1};
    int **pp = refs;
    int x = **pp++;
    int y = *(*pp + 1);

    **pp += y;
    printf("%d %d %d %td", x, y, a[0], pp - refs);
    return 0;
}
```

- A. 6 2 4 0

- B. 2 4 6 1
- C. 6 4 6 1
- D. 6 4 4 2

16. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[] = {1, 2, 3, 4, 5};
    int *p = a;
    int sum = 0;

    while (p < a + 5) {
        int x = *p++;
        if (x % 2 != 0)
            sum += x;
    }

    printf("%d %td", sum, p - a);
    return 0;
}
```

- A. 6 5
- B. 9 5
- C. 9 4
- D. 15 5

17. 下列程序的输出是：

```
#include <stdio.h>

int same_size(int a[9])
{
    return sizeof a == sizeof(int *);
}

int main(void)
{
    int a[9] = {0};

    printf("%d %d",
           same_size(a),
           sizeof a == 9 * sizeof a[0]);
    return 0;
}
```

- A. 0 0
- B. 0 1
- C. 1 1
- D. 1 0

18. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[3][4] = {
        {1, 3, 5, 7},
        {2, 4, 6, 8},
        {10, 20, 30, 40}
    };
    int (*p)[4] = a + 1;

    (*p)[2] += p[-1][1];
    p++;
    printf("%zu %d %d",
        sizeof *p / sizeof **p,
        p[-1][2], p[0][0]);
    return 0;
}
```

- A. 3 9 10
- B. 4 6 10
- C. 4 9 20
- D. 4 9 10

五、链表与二级指针综合

第19—32题及第36—38题默认已包含 `stdio.h`、`stdlib.h`，并使用以下结点类型；若无特别说明，动态内存申请均成功。

```
typedef struct Node {
    int data;
    struct Node *next;
} Node;
```

19. 下列程序的输出是：

```
Node **first_negative_slot(Node **head)
{
    while (*head != NULL && (*head)->data >= 0)
        head = &(*head)->next;
    return head;
}

int main(void)
{
    Node b = {6, NULL}, a = {4, &b}, *p;
    Node *head = &a;
    Node **slot = first_negative_slot(&head);
    Node *n = malloc(sizeof *n);

    n->data = -1;
    n->next = *slot;
    *slot = n;
    for (p = head; p != NULL; p = p->next)
        printf("%d ", p->data);
    return 0;
}
```

- A. -1 4 6
- B. 4 6 -1
- C. 4 -1 6
- D. 空表

20. 下列程序的输出是：

```
int swap_with_next(Node **head, int x)
{
    Node **link = head;
    Node *first, *second;

    while (*link != NULL && (*link)->data != x)
        link = &(*link)->next;
    if (*link == NULL || (*link)->next == NULL)
        return 0;
    first = *link;
    second = first->next;
    first->next = second->next;
    second->next = first;
    *link = second;
    return 1;
}

int main(void)
{
    Node d = {4, NULL}, c = {3, &d};
    Node b = {2, &c}, a = {1, &b};
    Node *head = &a, *p;
    int ok;

    ok = swap_with_next(&head, 2);
    printf("%d|", ok);
    for (p = head; p != NULL; p = p->next)
        printf("%d", p->data);
    return 0;
}
```

- A. 1|1234
- B. 0|1234
- C. 1|1324
- D. 1|1342

21. 下列程序的输出是：

```
void partition(Node *head, int pivot, Node **low, Node **high)
{
    Node **lt = low, **ht = high;

    *low = NULL;
    *high = NULL;
    while (head != NULL) {
        Node *next = head->next;
        head->next = NULL;
        if (head->data < pivot) {
            *lt = head;
            lt = &head->next;
        } else {
            *ht = head;
            ht = &head->next;
        }
        head = next;
    }
}

void show(Node *p)
{
    while (p != NULL) {
        printf("%d", p->data);
        p = p->next;
    }
}

int main(void)
{
    Node e = {3, NULL}, d = {2, &e}, c = {4, &d};
    Node b = {1, &c}, a = {5, &b};
    Node *low, *high;

    partition(&a, 3, &low, &high);
    show(low); putchar('|'); show(high);
    return 0;
}
```

- A. 21|345

- B. $12|345$
- C. $12|543$
- D. $543|12$

22. 下列程序的输出是：

```
void split_middle(Node *head, Node **front, Node **back)
{
    Node *slow, *fast;

    if (head == NULL) {
        *front = *back = NULL;
        return;
    }
    slow = head;
    fast = head->next;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    *front = head;
    *back = slow->next;
    slow->next = NULL;
}

void show(Node *p)
{
    while (p != NULL) {
        printf("%d", p->data);
        p = p->next;
    }
}

int main(void)
{
    Node f = {6, NULL}, e = {5, &f}, d = {4, &e};
    Node c = {3, &d}, b = {2, &c}, a = {1, &b};
    Node *front, *back;

    split_middle(&a, &front, &back);
    show(front); putchar('|'); show(back);
    return 0;
}
```

- A. 123|456
- B. 12|3456

- C. 1234|56
- D. 456|123

23. 下列程序的输出是：

```
Node *zip(Node *a, Node *b)
{
    Node dummy = {0, NULL};
    Node *tail = &dummy;

    while (a != NULL || b != NULL) {
        if (a != NULL) {
            tail->next = a;
            a = a->next;
            tail = tail->next;
        }
        if (b != NULL) {
            tail->next = b;
            b = b->next;
            tail = tail->next;
        }
    }
    tail->next = NULL;
    return dummy.next;
}

int main(void)
{
    Node a3 = {50, NULL}, a2 = {30, &a3}, a1 = {10, &a2};
    Node b3 = {60, NULL}, b2 = {40, &b3}, b1 = {20, &b2};
    Node *p = zip(&a1, &b1);

    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
    return 0;
}
```

- A. 10 30 50 20 40 60
- B. 10 20 30 40 50 60
- C. 20 10 40 30 60 50

- D. 10 20 40 30 50 60

24. 下列程序的输出是：

```
void choose_smaller_head(Node ***selected,
                        Node **left, Node **right)
{
    if ((*left)->data < (*right)->data)
        *selected = left;
    else
        *selected = right;
}

int main(void)
{
    Node l2 = {9, NULL}, l1 = {8, &l2};
    Node r2 = {4, NULL}, r1 = {3, &r2};
    Node *left = &l1, *right = &r1, *moved, *p;
    Node **selected = NULL;

    choose_smaller_head(&selected, &left, &right);
    moved = *selected;
    *selected = moved->next;
    moved->next = left;
    left = moved;

    for (p = left; p != NULL; p = p->next)
        printf("%d", p->data);
    putchar('|');
    for (p = right; p != NULL; p = p->next)
        printf("%d", p->data);
    return 0;
}
```

- A. 389|4
- B. 89|34
- C. 8349|空表
- D. 489|3

25. 下列程序的输出是：

```
int accumulate_from_tail(Node *p)
{
    int suffix;

    if (p == NULL)
        return 0;
    suffix = accumulate_from_tail(p->next);
    p->data += suffix;
    return p->data;
}

int main(void)
{
    Node c = {3, NULL}, b = {2, &c}, a = {1, &b};
    Node *p;
    int result = accumulate_from_tail(&a);

    printf("%d|", result);
    for (p = &a; p != NULL; p = p->next)
        printf("%d", p->data);
    return 0;
}
```

- A. 3|123
- B. 6|356
- C. 6|653
- D. 5|153

26. 下列程序的输出是：

```
void mark(Node *p, int depth)
{
    if (p == NULL)
        return;

    p->data += depth;
    mark(p->next, depth + 1);
    p->data += depth;
}

int main(void)
{
    Node c = {10, NULL}, b = {10, &c}, a = {10, &b};
    Node *p;

    mark(&a, 1);
    for (p = &a; p != NULL; p = p->next)
        printf("%d ", p->data);
    return 0;
}
```

- A. 12 14 16
- B. 11 12 13
- C. 16 14 12
- D. 12 13 14

27. 下列程序的输出是：

```
Node *swap_pairs(Node *head)
{
    Node *second;

    if (head == NULL || head->next == NULL)
        return head;
    second = head->next;
    head->next = swap_pairs(second->next);
    second->next = head;
    return second;
}

int main(void)
{
    Node e = {5, NULL}, d = {4, &e}, c = {3, &d};
    Node b = {2, &c}, a = {1, &b}, *p;
    Node *head = swap_pairs(&a);

    for (p = head; p != NULL; p = p->next)
        printf("%d", p->data);
    return 0;
}
```

- A. 54321
- B. 12345
- C. 21435
- D. 21543

28. 下列程序的输出是：

```
int length(Node *p)
{
    int n = 0;
    while (p != NULL) {
        n++;
        p = p->next;
    }
    return n;
}

Node *first_common(Node *a, Node *b)
{
    int la = length(a), lb = length(b);

    while (la > lb) { a = a->next; la--; }
    while (lb > la) { b = b->next; lb--; }
    while (a != b) {
        a = a->next;
        b = b->next;
    }
    return a;
}

int main(void)
{
    Node s2 = {11, NULL}, s1 = {9, &s2};
    Node a2 = {3, &s1}, a1 = {1, &a2};
    Node b1 = {2, &s1};
    Node *p = first_common(&a1, &b1);

    printf("%d", p == NULL ? -1 : p->data);
    return 0;
}
```

- A. 3
- B. 9
- C. 2
- D. -1

29. 下列程序的输出是：

```
Node *shared_address(Node *a, Node *b)
{
    Node *p, *q;

    for (p = a; p != NULL; p = p->next)
        for (q = b; q != NULL; q = q->next)
            if (p == q)
                return p;
    return NULL;
}

int main(void)
{
    Node a2 = {7, NULL}, a1 = {1, &a2};
    Node b2 = {7, NULL}, b1 = {2, &b2};
    Node *p = shared_address(&a1, &b1);

    printf("%d", p == NULL ? -1 : p->data);
    return 0;
}
```

- A. 1
- B. 2
- C. 7
- D. -1

30. 下列程序的输出是：

```
void rotate_tail(Node **tail, int steps)
{
    while (*tail != NULL && steps-- > 0)
        *tail = (*tail)->next;
}

int main(void)
{
    Node a = {1, NULL}, b = {2, NULL};
    Node c = {3, NULL}, d = {4, NULL};
    Node *tail = &d, *head, *p;

    a.next = &b; b.next = &c;
    c.next = &d; d.next = &a;
    rotate_tail(&tail, 2);
    head = tail->next;
    p = head;
    do {
        printf("%d", p->data);
        p = p->next;
    } while (p != head);
    return 0;
}
```

- A. 3412
- B. 1234
- C. 2341
- D. 4123

31. 下列程序的输出是：

```
void split_circle(Node *head, Node **first, Node **second)
{
    Node *slow = head, *fast = head;

    while (fast->next != head && fast->next->next != head) {
        slow = slow->next;
        fast = fast->next->next;
    }
    if (fast->next->next == head)
        fast = fast->next;
    *first = head;
    *second = slow->next;
    fast->next = *second;
    slow->next = *first;
}

int circle_sum(Node *head)
{
    Node *p = head;
    int sum = 0;
    do {
        sum += p->data;
        p = p->next;
    } while (p != head);
    return sum;
}

int main(void)
{
    Node a = {1, NULL}, b = {2, NULL}, c = {3, NULL};
    Node d = {4, NULL}, e = {5, NULL}, f = {6, NULL};
    Node *first, *second;

    a.next = &b; b.next = &c; c.next = &d;
    d.next = &e; e.next = &f; f.next = &a;
    split_circle(&a, &first, &second);
    printf("%d %d", circle_sum(first), circle_sum(second));
    return 0;
}
```

- A. 3 18
- B. 10 11
- C. 6 15
- D. 15 6

32. 下列程序的输出是：

```
int erase_from_circle(Node *head, int x)
{
    Node *p = head;
    int count = 0;

    while (p->next != head) {
        if (p->next->data == x) {
            Node *q = p->next;
            p->next = q->next;
            free(q);
            count++;
        } else {
            p = p->next;
        }
    }
    return count;
}

int main(void)
{
    Node head = {0, NULL};
    Node *a = malloc(sizeof *a), *b = malloc(sizeof *b);
    Node *c = malloc(sizeof *c), *d = malloc(sizeof *d), *p;
    int count, sum = 0;

    a->data = 2; a->next = b;
    b->data = 3; b->next = c;
    c->data = 2; c->next = d;
    d->data = 4; d->next = &head; head.next = a;
    count = erase_from_circle(&head, 2);
    for (p = head.next; p != &head; p = p->next)
        sum += p->data;
    printf("%d %d", count, sum);
    return 0;
}
```

- A. 1 9
- B. 2 7
- C. 2 5
- D. 3 7

第33—35题默认已包含 `stdio.h`、`stdlib.h`，并使用以下双向链表结点类型：

```
typedef struct DNode {  
    int data;  
    struct DNode *prev;  
    struct DNode *next;  
} DNode;
```

33. 下列程序的输出是：

```
void move_front(DNode **head, DNode *q)
{
    if (q == *head)
        return;

    q->prev->next = q->next;
    if (q->next != NULL)
        q->next->prev = q->prev;
    q->prev = NULL;
    q->next = *head;
    (*head)->prev = q;
    *head = q;
}

int main(void)
{
    DNode a = {1, NULL, NULL}, b = {2, NULL, NULL};
    DNode c = {3, NULL, NULL}, d = {4, NULL, NULL};
    DNode *head = &a, *tail = &d, *p;

    a.next = &b; b.prev = &a; b.next = &c;
    c.prev = &b; c.next = &d; d.prev = &c;
    move_front(&head, &c);
    for (p = head; p != NULL; p = p->next) printf("%d", p->data);
    putchar('|');
    for (p = tail; p != NULL; p = p->prev) printf("%d", p->data);
    return 0;
}
```

- A. 3124|4312
- B. 3124|4213
- C. 1234|4321
- D. 1324|4231

34. 下列程序的输出是：

```
DNode *reverse_dlist(DNode *head)
{
    DNode *p = head, *new_head = NULL;

    while (p != NULL) {
        DNode *next = p->next;
        p->next = p->prev;
        p->prev = next;
        new_head = p;
        p = next;
    }
    return new_head;
}

int main(void)
{
    DNode a = {1, NULL, NULL}, b = {2, NULL, NULL};
    DNode c = {3, NULL, NULL}, d = {4, NULL, NULL};
    DNode *head, *p;

    a.next = &b; b.prev = &a; b.next = &c;
    c.prev = &b; c.next = &d; d.prev = &c;
    head = reverse_dlist(&a);
    for (p = head; p != NULL; p = p->next) printf("%d", p->data);
    putchar('|');
    for (p = &a; p != NULL; p = p->prev) printf("%d", p->data);
    return 0;
}
```

- A. 1234|4321
- B. 4321|4321
- C. 1432|2341
- D. 4321|1234

35. 下列程序的输出是：

```
void detach(DNode *q)
{
    if (q->prev != NULL)
        q->prev->next = q->next;
    if (q->next != NULL)
        q->next->prev = q->prev;
    q->prev = NULL;
    q->next = NULL;
}

void append(DNode **head, DNode **tail, DNode *n)
{
    n->prev = *tail;
    n->next = NULL;
    if (*tail != NULL)
        (*tail)->next = n;
    else
        *head = n;
    *tail = n;
}

int main(void)
{
    DNode a = {1, NULL, NULL}, b = {2, NULL, NULL};
    DNode c = {3, NULL, NULL}, *head = &a, *tail = &c, *p;

    a.next = &b; b.prev = &a; b.next = &c; c.prev = &b;
    detach(&b);
    append(&head, &tail, &b);
    for (p = head; p != NULL; p = p->next) printf("%d", p->data);
    putchar('|');
    for (p = tail; p != NULL; p = p->prev) printf("%d", p->data);
    return 0;
}
```

- A. 132|231
- B. 123|321
- C. 312|213
- D. 132|321

36. 下列程序的输出是：

```
int pop_front(Node **head, int *value)
{
    Node *q;

    if (*head == NULL)
        return 0;
    q = *head;
    *value = q->data;
    *head = q->next;
    free(q);
    return 1;
}

int main(void)
{
    Node *head = malloc(sizeof *head);
    int value = -1, first, second;

    head->data = 7;
    head->next = NULL;
    first = pop_front(&head, &value);
    second = pop_front(&head, &value);
    printf("%d:%d:%d:%d", first, value, second, head == NULL);
    return 0;
}
```

- A. 1:7:1:1
- B. 0:-1:0:1
- C. 1:7:0:1
- D. 1:-1:0:0

37. 下列程序的输出是：

```
Node *clone(Node *p)
{
    Node *n;

    if (p == NULL)
        return NULL;
    n = malloc(sizeof *n);
    n->data = p->data;
    n->next = clone(p->next);
    return n;
}

void clear(Node **head)
{
    while (*head != NULL) {
        Node *q = *head;
        *head = q->next;
        free(q);
    }
}

int main(void)
{
    Node *original = malloc(sizeof *original);
    Node *second = malloc(sizeof *second), *copy;

    original->data = 1; original->next = second;
    second->data = 2; second->next = NULL;
    copy = clone(original);
    original->data = 9;
    printf("%d %d|", original->data, copy->data);
    clear(&original); clear(&copy);
    printf("%d %d", original == NULL, copy == NULL);
    return 0;
}
```

- A. 9 9|1 1
- B. 9 1|1 1
- C. 1 1|0 0
- D. 9 1|0 0

38. 下列程序的输出是：

```
int clear_circle(Node **tail)
{
    Node *p;
    int count = 0;

    if (*tail == NULL)
        return 0;
    p = (*tail)->next;
    (*tail)->next = NULL;
    *tail = NULL;
    while (p != NULL) {
        Node *next = p->next;
        free(p);
        p = next;
        count++;
    }
    return count;
}

int main(void)
{
    Node *a = malloc(sizeof *a), *b = malloc(sizeof *b);
    Node *c = malloc(sizeof *c), *d = malloc(sizeof *d);
    Node *tail = d;
    int count;

    a->data = 1; a->next = b;
    b->data = 2; b->next = c;
    c->data = 3; c->next = d;
    d->data = 4; d->next = a;
    count = clear_circle(&tail);
    printf("%d %d", count, tail == NULL);
    return 0;
}
```

- A. 0 1
- B. 4 0
- C. 1 4
- D. 4 1

六、指针表达式、函数与输入输出综合

39. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[] = {1, 2, 3, 4};
    int *p = a + 1;
    int *q = p++;

    (*q)++;
    printf("%d %d %td", a[1], *p, p - a);
    return 0;
}
```

- A. 2 3 2
- B. 3 2 1
- C. 3 3 2
- D. 2 2 1

40. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[] = {2, 4, 0, 8};
    int *p = a;
    int sum = 0;

    while (*p != 0)
        sum += *p++;
    printf("%d %td", sum, p - a);
    return 0;
}
```

- A. 6 3
- B. 6 2
- C. 14 4
- D. 2 1

41. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[] = {2, 5, 8, 11};
    int *p = a + 4;
    int sum = 0;

    while (p != a + 1)
        sum += *--p;
    printf("%d %td", sum, p - a);
    return 0;
}
```

- A. 26 0
- B. 19 2
- C. 24 0
- D. 24 1

42. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int x = 1, y = 5;
    int *refs[] = {&x, &y, &x};
    int i;

    for (i = 0; i < 3; i++)
        ++*refs[i];
    printf("%d %d %d", x, y, *refs[2]);
    return 0;
}
```

- A. 2 6 2
- B. 3 6 3
- C. 3 8 3
- D. 2 7 2

43. 下列程序的输出是：

```
#include <stdio.h>

void point_to_larger(int **out, int *x, int *y)
{
    *out = *x >= *y ? x : y;
}

int main(void)
{
    int x = 4, y = 9;
    int *p = NULL;

    point_to_larger(&p, &x, &y);
    *p -= 3;
    printf("%d %d %d", x, y, p == &y);
    return 0;
}
```

- A. 4 9 1
- B. 4 6 1
- C. 1 6 0
- D. 4 6 0

44. 下列程序的输出是：

```
#include <stdio.h>

void scale_column(int (*a)[3], int rows, int col)
{
    int i;

    for (i = 0; i < rows; i++)
        a[i][col] *= 2;
}

int main(void)
{
    int a[2][3] = {{1, 2, 3}, {4, 5, 6}};

    scale_column(a, 2, 1);
    printf("%d %d %d", a[0][1], a[1][1], *((a + 1) + 1));
    return 0;
}
```

- A. 2 5 5
- B. 4 5 10
- C. 2 10 10
- D. 4 10 10

45. 下列程序的输出是（动态内存申请成功）：

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int (*m)[3] = malloc(2 * sizeof *m);
    int i, j;

    for (i = 0; i < 2; i++)
        for (j = 0; j < 3; j++)
            m[i][j] = (i + 1) * (j + 1);

    printf("%d %d", m[1][2], *((m + 0) + 2));
    free(m);
    return 0;
}
```

- A. 3 6
- B. 6 2
- C. 6 3
- D. 3 2

46. 输入为一行 4 7 2 x 9。下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[5];
    int *p = a, *q;
    int sum = 0;

    while (p < a + 5 && scanf("%d", p) == 1)
        p++;
    for (q = a; q < p; q++)
        sum += *q;

    printf("%td %d", p - a, sum);
    return 0;
}
```

- A. 3 13
- B. 4 22
- C. 3 22
- D. 2 11

七、链表综合

第47—50题使用以下结点定义，且所需动态内存申请均成功：

```
typedef struct Node {
    int data;
    struct Node *next;
} Node;
```

47. 下列程序的输出是：

```
#include <stdio.h>
#include <stdlib.h>

void enqueue(Node **head, Node **tail, int x)
{
    Node *n = malloc(sizeof *n);

    n->data = x;
    n->next = NULL;
    if (*tail == NULL)
        *head = n;
    else
        (*tail)->next = n;
    *tail = n;
}

int dequeue(Node **head, Node **tail, int *value)
{
    Node *q;

    if (*head == NULL)
        return 0;
    q = *head;
    *value = q->data;
    *head = q->next;
    if (*head == NULL)
        *tail = NULL;
    free(q);
    return 1;
}

int main(void)
{
    Node *head = NULL, *tail = NULL;
    int first = -1, second = -1;

    enqueue(&head, &tail, 4);
    enqueue(&head, &tail, 6);
    dequeue(&head, &tail, &first);
    dequeue(&head, &tail, &second);
    enqueue(&head, &tail, 8);
```

```
    printf("%d %d %d %d %d",  
           first, second, head->data, tail->data, head == tail);  
    return 0;  
}
```

- A. 4 6 8 8 1
- B. 4 6 6 8 0
- C. 6 4 8 8 1
- D. 4 6 8 8 0

48. 下列程序的输出是（链表已按非递减顺序排列）：

```
#include <stdio.h>
#include <stdlib.h>

int remove_duplicates(Node *head)
{
    int count = 0;

    while (head != NULL && head->next != NULL) {
        if (head->data == head->next->data) {
            Node *q = head->next;
            head->next = q->next;
            free(q);
            count++;
        } else {
            head = head->next;
        }
    }
    return count;
}

int main(void)
{
    Node *a = malloc(sizeof *a), *b = malloc(sizeof *b);
    Node *c = malloc(sizeof *c), *d = malloc(sizeof *d);
    Node *e = malloc(sizeof *e), *f = malloc(sizeof *f), *p;
    int count;

    a->data = 1; a->next = b;
    b->data = 1; b->next = c;
    c->data = 1; c->next = d;
    d->data = 2; d->next = e;
    e->data = 2; e->next = f;
    f->data = 3; f->next = NULL;
    count = remove_duplicates(a);
    printf("%d|", count);
    for (p = a; p != NULL; p = p->next)
        printf("%d", p->data);
    return 0;
}
```

- A. 2|1123
- B. 3|123
- C. 3|1123
- D. 4|13

49. 下列程序的输出是：

```
#include <stdio.h>

void adjust(Node *p, int *sum)
{
    if (p->data % 2 == 0)
        p->data *= 2;
    *sum += p->data;
}

void for_each(Node *head, void (*visit)(Node *, int *), int *sum)
{
    while (head != NULL) {
        visit(head, sum);
        head = head->next;
    }
}

int main(void)
{
    Node d = {4, NULL}, c = {3, &d};
    Node b = {2, &c}, a = {1, &b}, *p;
    int sum = 0;

    for_each(&a, adjust, &sum);
    printf("%d|", sum);
    for (p = &a; p != NULL; p = p->next)
        printf("%d", p->data);
    return 0;
}
```

- A. 10|1234
- B. 16|1234
- C. 16|1438
- D. 12|1438

50. 下列程序的输出是：

```
#include <stdio.h>

Node *rotate_left(Node *head)
{
    Node *first, *tail;

    if (head == NULL || head->next == NULL)
        return head;
    first = head;
    head = head->next;
    tail = head;
    while (tail->next != NULL)
        tail = tail->next;
    tail->next = first;
    first->next = NULL;
    return head;
}

int main(void)
{
    Node d = {4, NULL}, c = {3, &d};
    Node b = {2, &c}, a = {1, &b};
    Node *p = rotate_left(&a);

    while (p != NULL) {
        printf("%d", p->data);
        p = p->next;
    }
    return 0;
}
```

- A. 2341
- B. 1234
- C. 4123
- D. 2314

答题卡

1. __	2. __	3. __	4. __	5. __
6. __	7. __	8. __	9. __	10. __
11. __	12. __	13. __	14. __	15. __
16. __	17. __	18. __	19. __	20. __
21. __	22. __	23. __	24. __	25. __
26. __	27. __	28. __	29. __	30. __
31. __	32. __	33. __	34. __	35. __
36. __	37. __	38. __	39. __	40. __
41. __	42. __	43. __	44. __	45. __
46. __	47. __	48. __	49. __	50. __

完成后可发送填写好的答案或清晰照片进行批改。